

POS Transaction Integrity — What We Changed & What You Need To Do

Audience: ECAPS (Equity Capital) integration & operations team · **Prepared by:** Sameday Solution Engineering

Purpose: Ensure that a POS `SUCCESS` you receive is safe to settle to your end-users instantly, and that any transaction that fails or reverses is communicated to you reliably and within minutes.

1. The problem we solved

Pine Labs' transaction API reports a card swipe as `SUCCESS` at the moment it is **authorized at the terminal**. A small fraction of these are later **auto-reversed** (e.g. a terminal timeout causes the card to be auto-refunded). Because you settle your end-users **instantly** on our `SUCCESS` callback, a late reversal previously meant you had already paid a user for a transaction that ultimately failed — with no clean way to recover.

We have shipped a layered fix so that:

1. A `SUCCESS` we send you is **confirmed**, not merely authorized.
2. If a transaction does reverse, you are told **within minutes**, not hours.
3. A financial **safety reserve** backs the rare residual case.

2. What Sameday has implemented

Layer 1 Confirmation hold before `SUCCESS` — catches ~99%

We no longer forward a `SUCCESS` the instant we see it. A fresh success is held for a short confirmation window (default **120 seconds**) and re-checked against Pine Labs' live status. If it auto-reverses inside that window, **we never send you a `SUCCESS` at all**. To you this remains effectively real-time (seconds), but the success is now trustworthy.

Layer 2 Fast reversal notifications — catches the slip-through

For anything that reverses **after** we've sent `SUCCESS`, a dedicated fast-reconcile job re-checks recent transactions every **~5 minutes** and immediately emits a `pos.transaction.reversed` webhook. The faster you receive it, the more likely the funds are still in your user's wallet and can be recovered cleanly.

Layer 3 Rolling reserve — financial backstop

A small, configurable percentage of each settlement is held back into a **rolling reserve** for a few days. If a settled transaction later reverses, the loss is **automatically drawn down from the reserve** instead of requiring a manual clawback. Matured holds are released back automatically. This converts a rare, unpredictable loss into a small, managed buffer.

Layer 4 Safe payout gating — active on our side

We never pay out on a transaction that reconciliation has flipped to failed/void/refund, and we require positive Pine Labs settlement confirmation before releasing funds to you.

3. What you (ECAPS) need to do

These are the actions required on **your** side to make the above effective:

1. **Subscribe to and process the `pos.transaction.reversed` event.** Your webhook endpoint must handle **two** event types:

- `pos.transaction` — a confirmed sale. Safe to settle to your user.
- `pos.transaction.reversed` — a previously-reported sale that failed/reversed. On receipt, **reverse the corresponding credit** to your end-user (or claw it back).

Confirm your webhook subscription includes the `pos` event category so both are delivered.

2. **Settle your end-users on `pos.transaction` only** (the confirmed event) — not on any earlier/provisional signal. With Layer 1 in place, this event is now safe.

3. **Handle the reversal idempotently and promptly.** Match on `txn_id` (and `rrn`) and reverse exactly once.

Example payload:

```
{
  "event": "pos.transaction.reversed",
  "action": "remove",
  "txn_id": "PL_7280020571",
  "rrn": "626819668379",
  "tid": "43159304",
  "mid": "KKE379",
  "amount": 1000,
  "previous_status": "CAPTURED",
  "status": "FAILED",
  "reversed_at": "2026-09-25T14:12:10.000Z",
  "reason": "pinelab-recon:FAILED",
  "_brand": "PINELAB"
}
```

4. **Verify webhook signatures.** Every callback is signed:

- `X-Sameday-Signature` : `HMAC-SHA256(webhook_secret, "{timestamp}.{rawBody}")` (hex)
- `X-Sameday-Timestamp` : unix seconds (reject if too old)
- `X-Sameday-Delivery` : per-delivery id (use as idempotency key)
- `X-Sameday-Event` : the event type

5. **Acknowledge with a 2xx.** Non-2xx responses are retried; a persistently failing endpoint delays delivery.

4. Net effect

Scenario	Before	After
Swipe auto-reverses within seconds	You got <code>SUCCESS</code> , settled user, ate the loss	You never receive <code>SUCCESS</code> — nothing to reverse
Swipe reverses after we sent <code>SUCCESS</code>	No / late failure signal	<code>pos.transaction.reversed</code> within minutes → you claw back
Reversal after you already paid & user withdrew	Unrecoverable loss	Covered automatically by the rolling reserve

Bottom line: your settlement to users stays instant, and a `SUCCESS` from us is now a signal you can safely act on.